

1. Create a new branch.
2. Add new code in the branch.
3. Commit the changes.
4. Push the branch to GitHub.
5. Create a Pull Request (PR).
6. Merge the Pull Request into the main branch.
7. Pull the updated main branch.

**Important:** A **Pull Request (PR)** is a feature of **GitHub**, not Git itself. So, to demonstrate a PR, you'll need a GitHub account and a remote repository.

---

## Practical B: Demonstrate Branching and Merging Using Pull Requests

### Prerequisites

Before starting, make sure you have:

- ✓ Git installed
  - ✓ VS Code installed
  - ✓ GitHub account
  - ✓ Completed Part (a)
  - ✓ Your project (Git\_Practical) already committed
- 

### Current Project Structure

```
Git_Practical
├── hello.py
├── README.md
└── .git
```

---

## Step 1: Open Your Project

Open **VS Code**.

Go to:

**File** → **Open Folder** → **Git\_Practical**

Open the terminal:

**Terminal** → **New Terminal**

---

## Step 2: Check the Current Branch

Type:

```
git branch
```

Output:

```
* master
```

or

```
* main
```

If your repository shows **master**, use `master` in the commands below instead of `main`.

---

## Step 3: Rename `master` to `main` (Optional)

Modern repositories use `main`.

If your branch is `master`, type:

```
git branch -M main
```

Check again:

```
git branch
```

Output:

```
* main
```

---

## Step 4: Create a New Feature Branch

Create a branch called **feature-login**:

```
git checkout -b feature-login
```

Output:

```
Switched to a new branch 'feature-login'
```

---

## Step 5: Verify the Branch

```
git branch
```

Output:

```
* feature-login  
main
```

The \* indicates the current branch.

---

## Step 6: Modify the Python File

Open:

```
hello.py
```

Current code:

```
print("Hello Git!")  
print("Welcome to Git Repository")
```

Add new lines:

```
print("Hello Git!")  
print("Welcome to Git Repository")  
  
print("Login Feature Added")  
print("Feature Branch Example")
```

Save the file (**Ctrl + S**).

---

## Step 7: Check Status

```
git status
```

Output:

```
modified: hello.py
```

---

## Step 8: Stage the Changes

```
git add hello.py
```

---

## Step 9: Commit the Changes

```
git commit -m "Added login feature"
```

Example output:

```
[feature-login abc123]
```

```
1 file changed
```

---

## Step 10: View Commit History

```
git log --oneline
```

Example:

```
7b8c91 Added login feature  
2a3bc4 Updated hello.py  
1f23de Initial Python Project
```

---

## Step 11: Create a Repository on GitHub

1. Open GitHub.
2. Click **New Repository**.
3. Repository name:

```
Git_Practical
```

4. Keep it **Public**.
5. Do **not** add a README (you already have one).
6. Click **Create Repository**.

---

## Step 12: Connect Local Repository to GitHub

Copy your GitHub repository URL, for example:

```
https://github.com/yourusername/Git_Practical.git
```

Run:

```
git remote add origin https://github.com/yourusername/Git_Practical.git
```

Verify:

```
git remote -v
```

Output:

```
origin https://github.com/yourusername/Git_Practical.git
```

---

## Step 13: Push the Main Branch

Switch to the main branch:

```
git checkout main
```

Push it:

```
git push -u origin main
```

---

## Step 14: Push the Feature Branch

Switch back:

```
git checkout feature-login
```

Push:

```
git push -u origin feature-login
```

Output:

Branch 'feature-login' set up to track remote branch.

---

## Step 15: Create a Pull Request

Go to GitHub.

You will see:

Compare & Pull Request

Click it.

---

## Step 16: Fill Pull Request Details

Title:

Added Login Feature

Description:

This pull request adds a login feature to the Python project.

Click:

Create Pull Request

---

## Step 17: Merge the Pull Request

Click:

Merge Pull Request

Then:

Confirm Merge

GitHub will display:

Pull Request Successfully Merged

---

## Step 18: Switch Back to Main

In VS Code:

```
git checkout main
```

---

## Step 19: Pull the Latest Changes

```
git pull origin main
```

Output:

```
Updating...  
Fast-forward
```

---

## Step 20: Verify the Merge

Open:

```
hello.py
```

You should see:

```
print("Hello Git!")  
print("Welcome to Git Repository")  
  
print("Login Feature Added")  
print("Feature Branch Example")
```

The feature branch changes are now in the main branch.

---

## Step 21: View Branches

```
git branch
```

Output:

```
feature-login
```

\* main

---

## Step 22: Delete the Feature Branch (Optional)

Delete the local branch:

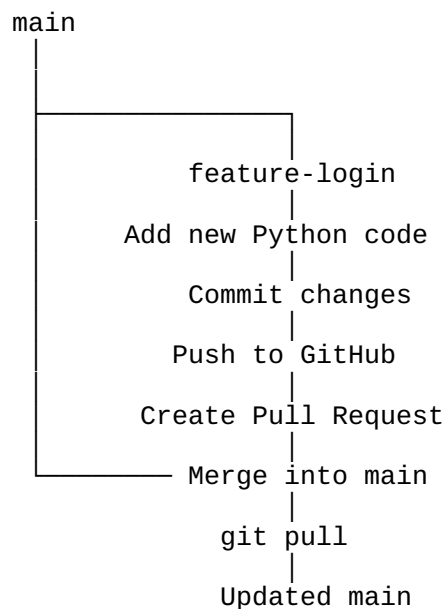
```
git branch -d feature-login
```

Delete the remote branch:

```
git push origin --delete feature-login
```

---

## Workflow Diagram



## Git Commands Used

| Command                                    | Purpose                           |
|--------------------------------------------|-----------------------------------|
| <code>git branch</code>                    | Display branches                  |
| <code>git checkout -b feature-login</code> | Create and switch to a new branch |
| <code>git add hello.py</code>              | Stage the modified file           |

| Command                                                   | Purpose                            |
|-----------------------------------------------------------|------------------------------------|
| <code>git commit -m "Added login feature"</code>          | Commit the changes                 |
| <code>git remote add origin &lt;repository-url&gt;</code> | Connect local repository to GitHub |
| <code>git push -u origin main</code>                      | Push the main branch               |
| <code>git push -u origin feature-login</code>             | Push the feature branch            |
| <code>git checkout main</code>                            | Switch to the main branch          |
| <code>git pull origin main</code>                         | Download merged changes            |
| <code>git branch -d feature-login</code>                  | Delete the local feature branch    |

---

## Viva Questions

### Q1. What is a Git branch?

**Answer:** A Git branch is a separate line of development that allows developers to work on new features or bug fixes without affecting the main code.

### Q2. Why is branching used?

**Answer:** Branching allows developers to develop and test features independently before merging them into the main project.

### Q3. What is a Pull Request (PR)?

**Answer:** A Pull Request is a GitHub feature used to propose changes from one branch to another. It allows code review, discussion, and approval before merging.

### Q4. What is the purpose of merging?

**Answer:** Merging combines changes from one branch into another, usually from a feature branch into the main branch.

### Q5. What is the difference between Git and GitHub?

**Answer:** Git is a version control system used locally to track changes, while GitHub is a cloud-based platform that hosts Git repositories and provides collaboration features such as Pull Requests.

This completes **Part (b): Demonstrate branching by creating a new feature branch, adding code, and merging it back into the main branch using Pull Requests** in a detailed, beginner-friendly manner.

## Step 11.1: Sign in to GitHub

1. Open your web browser (Chrome, Edge, Firefox, etc.).
2. Go to <https://github.com>.
3. Enter your **username/email** and **password**.
4. Click **Sign in**.

